BufferLab Analytics Logic Deep Dive

This document provides a complete, end-to-end explanation of how App2 (BufferLab) computes every metric and visualization once gold data is loaded and the data contract passes. It is intended to be sufficient for a reader to understand inputs, transformations, calculations, and outputs without reading the source code.

---

0. End-to-end flow (high level)

1. Load gold tables from `data/gold` into DuckDB (in-memory).
2. Validate the data contract (tables, columns, date types, required fields).
3. Build square sets and explode requirements (domain-based BOM logic).
4. Build the time-phased netting ledger (site/item/week inventory, arrivals, required).
5. Run pegging allocation (priority and demand tier waterfall, convergence gating).
6. Derive blockers, stranded inventory, segmentation, buffers, and scenario summaries.
7. Render UI pages and exports from the computed outputs.

Everything below provides the exact logic, inputs, and formulas behind those steps.

---

1. Inputs and required data

1.1 Required gold tables (must exist)

- `deployment_plan` (preferred) or `demand_plan` (fallback for plan)
- `bom_kit`
- `inventory_position`
- `supply`
- `site_readiness`
- `item_master`
- `node_master`
- `lane_master`

If `deployment_plan` is missing and `demand_plan` exists, the plan uses `demand_plan` as a fallback.

1.2 Optional gold tables (used if present)

- `square_set_master` (explicit square set mapping)
- `lead_time_history`
- `lead_time_distribution`
- `lifecycle`
- `substitution_map`

1.3 Required columns by table (core fields used in calculations)

`deployment_plan` (plan table):

- `week`, `site_id`, `kit_id`, `kits_planned`, `demand_tier` (optional but used if present)
- `priority` (optional), `program_id` (optional)

`demand_plan` (for buffer sizing and fallback):

- `week`, `site_id`, `kit_id`, `kits_planned`
- If item-level: `item_id`, `qty`, and `demand_tier` or `demand_type`

`bom_kit`:

- kit_id, child_item_id, qty_per, effective_start_week
- Optional: effective_end_week, kit_criticality

inventory_position:

- as_of_date, item_id, node_id, on_hand, usable_on_hand
- Optional: aging_days, unit_cost

supply:

- item_id, node_id, qty, status
- Must include one of: promised_date, promise_date, or promise_week
- Optional: allocation_flag, confidence_weight

site_readiness:

- scenario_id, site_id, week
- Optional: readiness_capacity_kits, power_ready_mw, readiness_state

item_master:

- item_id, category, subcategory, value_density, shared_flag, build_ahead_flag
- Optional: unit_cost, description, break_glass_exception

node_master:

- node_id, site_id, node_type, optional region

lane_master:

- from_node_id, to_node_id, transfer_lead_time_days
- Optional: transfer_capacity_units_per_week

lifecycle (optional):

- item_id, generation, compatibility_group, transition_start_date, transition_end_date, ltb_date, eol_date

lead_time_history / lead_time_distribution (optional):

- lead_time_p95 or p95 or lead_time_days (depending on table)

substitution_map (optional):

- `from_item_id`, `to_item_id`, `substitution_type`, `approval_required`

1.4 Required data types

- All date-like columns (`week`, `as_of_date`, `promised_date`, etc.) must be DATE or TIMESTAMP.
- Data contract validation will surface type mismatches.

1.5 Plan table selection

`deployment_plan` is used if present. If not present and `demand_plan` exists, `demand_plan` is used as the plan table.

---

2. Configuration defaults that affect analytics

Configuration lives in `configs/default_config.yml` and optional overrides in `configs/user_settings.yml`.

Key settings that change logic:

- `analysis.week_start`: week boundary (default Monday)
- `analysis.default_scenario`: used when no scenario is selected
- `analysis.horizon_weeks`: number of weeks to project in the ledger
- `analysis.transfers.enabled`: whether to model upstream transfers
- `analysis.transfers.assume_unlimited_capacity`: if true, ignores lane capacity
- `analysis.pegging.default_priority`: used if priority missing in plan
- `mw_per_kit.default`: used when power readiness is in MW
- `buffer_policy`: v1 buffer heuristic thresholds (used in /buffers page)
- Segmentation thresholds (from /settings UI)

---

3. Square set model (unit of deployment)

3.1 Square set master

If `square_set_master` exists, it is used directly. Otherwise, it is generated on the fly:

- Join `bom_kit` with `item_master` to infer kit domain by dominant item category.
- Map categories into domains using `DOMAIN_CATEGORIES`:
  - it_rack: GPU_IT_Rack, Rack, Server, Compute, Accelerator
  - callan: Callan, HXU, Cooling, Power, Sidecar
  - mor: MOR, Network, Front_End_Network, FEN, Switch, Optics, Cable
- For each site, assign kits to domains and build `square_set_master` with columns:
  - `square_set_id`, `site_id`, `it_rack_kit_id`, `callan_kit_id`, `mor_kit_id`, `power_mw_required`

3.2 Deployable kits in the plan (readiness integration)

For each row in the plan table:

```
plan_deployable = min(kits_planned, readiness_capacity)
```

`readiness_capacity` is computed as:

- If `readiness_capacity_kits` exists, use it.
- Else if `power_ready_mw` exists, use `power_ready_mw / mw_per_kit`.
- Else default to `kits_planned`.

Readiness is filtered by `scenario_id`.

3.3 Exploding square sets into item requirements

For each domain (it_rack, callan, mor):

- Join plan rows to `square_set_master` on `site_id` and domain kit id.
- Join to `bom_kit` on `kit_id`.
- Filter to blocking components only (`kit_criticality == 'blocking'` or NULL).
- Required quantity:

```
required_qty = sum(deployable_sets * qty_per)
```

This yields requirements by `(week, site_id, square_set_id, domain, item_id)`.

---

4. Demand tiers and priority logic

4.1 Demand tiers

- If `demand_tier` exists in the plan table, it is used.
- Otherwise, all demand is treated as `committed`.

For item-level demand in `demand_plan`, `demand_type` is mapped to tiers:

- committed: committed, firm, booked
- likely: likely, probable, forecast
- exploratory: anything else

4.2 Priority

- If `priority` exists in plan, it is used.
- Otherwise `analysis.pegging.default_priority` is used.

---

5. Supply, transfers, and time alignment

5.1 Supply week alignment

Supply uses one of `promised_date`, `promise_date`, or `promise_week` to align receipts to weeks.

5.2 Transfer model (multi-echelon)

If transfers are enabled and `lane_master` is present:

- Direct supply at site nodes contributes to arrivals at the site week.
- Upstream inventory (integration/regional) is treated as arriving at the minimum week.
- Upstream supply is shifted by `transfer_lead_time_days` to the site.
- Capacity is limited by `transfer_capacity_units_per_week` unless `assume_unlimited_capacity` is true.

If transfers are disabled or `lane_master` missing, only direct site supply is used.

---

6. Netting ledger (time-phased inventory)

The ledger is built per `(week, site_id, item_id)` and enforces no double counting.

Inputs:

- `required` from square set requirements
- `arrivals` from transfer model (site + upstream)
- `opening_from_inv` from `inventory_position` (only in first week)

Running balances per site/item across weeks:

```
opening_balance[t] = closing_balance[t-1] (or opening_from_inv for first week)
available[t] = opening_balance[t] + arrivals[t]
allocated[t] = min(required[t], max(available[t], 0))
closing_balance[t] = available[t] - allocated[t]
shortfall[t] = required[t] - allocated[t]
```

Outputs are used for pegging, constrained items, and stranded inventory.

---

7. Pegging (allocation) and convergence gating

7.1 Convergence gating

Before allocation, square sets are checked for convergence (all domains ready). If not converged, they are blocked before allocation.

A square set is fully ready if:

```
IT_ready AND Callan_ready AND MOR_ready
```

`power_ready_mw` is an additional gate if present.

7.2 Greedy allocation by priority (within demand tier)

For each tier in order `committed`, `likely`, `exploratory`:

1. Group requirements by `(week, site_id, square_set_id, priority)`.
2. Sort by `(week, site_id, priority)` ascending.
3. For each square set, compute max buildable based on available inventory:

```
item_can_build = available / qty_per
max_buildable = min(item_can_build for blocking items)
```

4. `buildable_sets = int(max_buildable)`
5. `blocked_sets = deployable_sets - buildable_sets`
6. Consume inventory for built sets only.

Output columns include: `deployable_sets`, `buildable_sets`, `blocked_sets`, `blocking_items`.

---

8. Blockers: what prevents readiness, where, and when

Blocker attribution is computed per blocked square set.

Inputs:

- Square set requirements (blocking items)
- Site inventory and upstream inventory (from `inventory_position` + `node_master`)
- Future supply at site and upstream (from `supply`)

Gap and root cause logic:

```
gap_qty = required_qty - available_now
if available_now >= required_qty: root_cause = 'no_gap'
else if available_now + upstream_qty >= required_qty: root_cause = 'transfer_delay'
else if available_now + upstream_qty + future_arriving >= required_qty: root_cause
= 'supply_timing'
else: root_cause = 'pure_shortage'
```

Additional attributes:

- `gap_value = gap_qty * unit_cost`
- Category/subcategory from `item_master`

Outputs:

- Detailed blocker list
- Pareto ranking by gap
- Fix recommendations (where available)

---

9. Stranded inventory and E&O exposure

Stranded inventory is inventory that is available but cannot be consumed because a square set is blocked.

Steps:

1. Identify blocked square sets from pegging (`blocked_sets > 0`).
2. Identify items in blocked square sets (blocking items only).
3. Identify partially ready domains: items where a domain is ready but full convergence fails.
4. Join the ledger closing balance with blocked items and keep positive balances.

Stranded value:

```
stranded_units = closing_balance
stranded_value = stranded_units * unit_cost
```

Also captures:

- `stranding_reasons` (blocked_set, partial_domain_ready)
- `blocked_by_item` and `blocked_by_cause` (top blocker per site/week)
- Aging days from `inventory_position` if present

---

10. Segmentation (MECE) and overlay tags

Segmentation classifies each `item_id` into exactly one base segment using three dimensions:

10.1 Blocker

```
is_blocker = (kit_criticality == 'blocking' OR kit_criticality IS NULL)
```

10.2 Constrained

```
is_constrained = allocation_flag
    OR avg(confidence_score/confidence_weight) < constrained_confidence
```

```
    OR lead_time_p95 > constrained_lead_time
```

Lead time is derived from `lead_time_distribution` or `lead_time_history` (p95 or quantile from lead_time_days).

10.3 High E&O

```
is_high_eo = (unit_cost > high_eo_unit_cost) AND (days_to_risk <
high_eo_days_to_risk)
```

If `use_category_relative_cost` is enabled, high cost is based on category percentile rather than absolute unit cost.

10.4 Base segments

Segments are assigned by the boolean cube of (blocker, constrained, high_eo):

- B1, B2, B3, B4 (blockers)
- N1, N2, N3, N4 (non-blockers)

10.5 Overlay tags

Non-exclusive tags applied after base segment:

- `transition_active`: today is between `transition_start_date` and `transition_end_date` (lifecycle)
- `shared_component`: item appears in >= `shared_usage_threshold` kits OR `shared_flag == True`
- `long_lead_foundation`: `lead_time_p95 > long_lead_threshold` AND `days_to_risk > long_lead_days_to_risk_min`
- `build_ahead_sensitivity`: stranded_ratio > `build_ahead_stranding_pct` OR `build_ahead_flag == True`
- `break_glass_exception`: `break_glass_exception == True`

Fungibility (from `substitution_map`) is calculated and attached for buffer adjustments.

---

11. Buffer targets (policy logic)

Two engines exist:

- v1 policy engine used in the Buffers page (`/buffers`)
- v2 segmentation-based engine used in exports and buffer analysis

11.1 Buffer Engine v1 (policy heuristic)

Inputs:

- Item category and kit criticality from `bom_kit` + `item_master`
- Supply risk from `supply` (allocation_flag, confidence_weight, lead_time)
- Lifecycle risk from `lifecycle`

Supply risk:

```
if allocation_flag OR avg_confidence < low_confidence_threshold OR lead_time_p95 >=
lead_time_high_risk: high
else if lead_time_p95 >= lead_time_medium_risk: medium
else: low
```

Lifecycle risk:

```
days_to_risk = min(eol_date, ltb_date) - today
risk = high if days_to_risk <= aging_critical
risk = medium if days_to_risk <= aging_warning
risk = low otherwise
```

Segment key:

```
category | kit_criticality | supply_risk | lifecycle_risk
```

Targets are looked up in `buffer_policy.targets` with category mapping (GPU, Callan, FEN, default).

11.2 Buffer Engine v2 (segmentation-based)

Target inventory is derived from the segment and demand tier.

Base ranges (min, max weeks) by segment:

- B1: 2-3 (integration)
- B2: 3-4 (integration)
- B3: 2-3 (regional)
- B4: 4-6 (regional)
- N1..N4: 1-2 (site)

Tier rules:

- committed: use max weeks (capped by committed_max_coverage_weeks)
- likely: use min weeks (capped by likely_max_coverage_weeks)
- exploratory: 0 weeks

Penalties and reductions:

- transition_active: reduce weeks by transition_buffer_reduction_pct
- high E&O segments (B1/B3/N1/N3): reduce max weeks by 25 percent
- substitution_type == minor_gen: reduce weeks by 15 percent

Target qty per item:

```
avg_weekly_demand = mean weekly demand (from demand_plan if item-level, else from
requirements)
min_buffer_qty = avg_weekly_demand * min_weeks
max_buffer_qty = avg_weekly_demand * max_weeks
buffer_target_qty = avg_weekly_demand * target_weeks
```

Value at risk (E&O exposure) used in buffer analysis:

```
inventory_days = current_inventory / avg_daily_demand
risk_multiplier = days_to_risk_factor * (365 / max(days_to_risk, 1))
value_at_risk = unit_cost * inventory_days * risk_multiplier
```

---

12. Scenario compare

Scenarios are defined by `site_readiness.scenario_id` values. The Scenarios page compares:

- completion rate
- blocked sets
- stranded value
- top blocker

Templates are derived from Scenario A using multipliers:

- baseline: 1.0 power, 1.0 supply, 1.0 demand
- favorable: +20% power, +10% supply, +0% demand
- stressed: -20% power, -20% supply, +20% demand

---

13. Cost/benefit view for leaders

The app exposes economic tradeoffs through:

1. **Stranded capital** (from Stranded Engine):
    - Dollar value of inventory blocked by missing components
    - Highlights where capital is tied up by readiness issues
2. **E&O exposure (value at risk)**:
    - Calculated in Buffer Engine v2 using unit cost, inventory days, and time-to-risk
3. **E&O penalty impact**:
    - Estimates buffer reductions and cost savings for high-risk items
4. **Scenario deltas**:
    - Scenario compare shows delta completion, delta blocked, delta stranded
5. **Buffer analysis export** (`/export/buffer-analysis`):
    - By segment and by location summaries
    - E&O penalty impact (items with penalty, buffer reduction, estimated savings)
    - Item-level buffer targets with value-at-risk fields

14. Outputs mapped to pages and exports

Overview (`/`)

- Completion rate, blocked sets, top blocker, stranded value
- Completion trend (weekly) and blocked sets by week

Readiness (`/readiness`)

- Planned vs deployable vs buildable by site/week
- Detail view of square sets and missing domains

Pegging (`/pegging`)

- Priority summary (P1/P2/P3 buckets)
- Pegging outcomes by square set

Blockers (`/blockers`)

- Top blockers with root causes and gap values
- Pareto chart of blocking items

Stranded (`/stranded`)

- Stranded units and value by site and item
- Stranding reasons and blocked-by attribution

Scenarios (`/scenarios`)

- Scenario comparison table and template variants

Convergence (`/convergence`)

- Domain readiness and convergence rate by week

Segments (`/segmentation`)

- B1-B4/N1-N4 counts and overlay tag counts
- Sample segmented items table

Buffers (`/buffers`)

- v1 policy buffer recommendations by segment

Exports

- `/export/csv/buffers` uses Buffer Engine v2 (item-level)
- `/export/csv/blockers`, `/export/csv/stranded`, `/export/csv/convergence`, `/export/csv/segments`, `/export/csv/pegging`
- `/export/buffer-analysis` for cost/benefit reporting
- `/export/weekly-status` leadership summary JSON

---

15. Assumptions, guardrails, and common edge cases

- If a required table is missing, the contract fails and most analytics are skipped.
- If dates are not DATE/TIMESTAMP, joins on `week` will fail or drop rows.
- If `square_set_master` is inconsistent with plan kits, requirements may be zero.
- If `site_readiness` lacks the selected scenario, readiness defaults to planned.
- If lead time or lifecycle data is missing, segmentation and buffer logic will still run but with reduced fidelity.

---

16. Summary

BufferLab converts gold data into an end-to-end readiness simulation by:

- Defining square sets as the deployable unit
- Exploding multi-domain BOM requirements
- Running a netting ledger with transfers and time-phased inventory
- Allocating supply by priority and demand tier
- Producing blockers, stranded value, segmentation, and buffer targets
- Quantifying economic tradeoffs via value at risk and buffer penalty impacts

This document covers the full analytics logic used to generate the application outputs once the gold data is present and valid.